

Praktik 1
CRUD Produk + SQLite

Praktik 2
Register & Login JWT

Praktik 3
Protected Routes

KERJAKAN SATU PER SATU — Selesaikan & test Praktek 1, Praktek 2 , setelah itu Praktek P3

P1 Selesai

CRUD produk jalan
5 endpoint OK

Test P1

Postman: GET, POST,
PUT, DELETE berhasil

P2 Selesai

Register & Login
JWT token OK

Test P2

Token diterima,
user tersimpan

P3 Selesai

authGuard aktif
401 tanpa token

Praktek harus berurut

- P1 setup awal project
- P2 menambah tabel User ke schema P1 — kalau P1 belum jalan, P2 pasti error
- P3 menambah authGuard ke routes P1 — kalau P2 belum ada token, P3 tidak bisa ditest
- Lebih mudah debug: kalau error muncul, kamu tahu persis di layer mana masalahnya
- Kamu bisa lihat hasilnya lebih cepat — tiap praktik sudah bisa ditest sendiri

Yang akan dibuat:

- Inisiasi project Node.js
- Database SQLite lewat Prisma ORM
- Model Product (id, nama, harga, stok)
- 5 endpoint: GET all, GET by id, POST, PUT, DELETE

Langkah-langkah P1:

- 1 Setup: mkdir, npm init, npm install
- 2 Buat .env dan .gitignore
- 3 Tulis schema.prisma (model Product)
- 4 Jalankan migration + buat db.js
- 5 Buat index.js (entry point)
- 6 Buat routes/products.js (CRUD)
- 7 Test semua endpoint di Postman

Struktur folder hasil P1:

```
produk-api/  
├── prisma/  
│   ├── schema.prisma  ← definisi tabel  
│   ├── dev.db          ← file database SQLite  
│   └── migrations/  
│       └── ../migration.sql  
├── routes/  
│   └── products.js     ← 5 CRUD endpoints  
├── db.js               ← koneksi Prisma  
├── index.js            ← entry point Express  
├── .env                ← DATABASE_URL, PORT  
└── .gitignore
```

Jalankan perintah ini di terminal, satu per satu:

```
# 1. Buat folder dan masuk ke dalamnya
mkdir produk-api
cd produk-api

# 2. Inisiasi npm (generate package.json)
npm init -y

# 3. Install semua dependency
npm install express dotenv @prisma/client bcryptjs jsonwebtoken
npm install -D prisma nodemon

# 4. Init Prisma dengan SQLite
npx prisma init --datasource-provider sqlite
```

Tambahkan scripts ke package.json:

```
"scripts": {
  "start": "node index.js",
  "dev": "nodemon index.js"
}
```

- npm init -y → buat package.json otomatis
- npm install → download semua package ke node_modules/
- prisma init → buat folder prisma/ dan file .env

Update .env

```
DATABASE_URL="file:./prisma/dev.db"
PORT=3000
JWT_SECRET="ganti_dengan_string_rahasia_panjang_min32karakter"
```

JWT_SECRET harus panjang & acak. Cara generate:

```
node -e
"console.log(require('crypto').randomBytes(32).toString('hex'))"
```

.gitignore

```
node_modules/
.env
prisma/dev.db
```

Tambah scripts ke package.json

```
{
  "name": "produk-api",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "start": "node index.js",
    "dev": "nodemon index.js"
  },
  "dependencies": {
    "express": "^4.x",
    "dotenv": "^16.x",
    "@prisma/client": "^5.x",
    "bcryptjs": "^2.x",
    "jsonwebtoken": "^9.x"
  },
  "devDependencies": {
    "nodemon": "^3.x",
    "prisma": "^5.x"
  }
}
```

prisma/schema.prisma

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "sqlite"
  url      = env("DATABASE_URL")
}

model Product {
  id          Int           @id @default(autoincrement())
  nama        String
  harga       Int
  stok        Int           @default(0)
  createdAt   DateTime      @default(now())
  updatedAt   DateTime      @updatedAt
}
```

Penjelasan anotasi:

@id

Primary key tabel — wajib ada di setiap model

@default(autoincrement())

ID otomatis: 1, 2, 3, ...

@default(0)

Nilai default jika tidak diisi saat create

@default(now())

Otomatis isi waktu sekarang

@updatedAt

Otomatis update waktu saat data di-edit

db.js (buat di root folder)

```
// db.js
const { PrismaClient } = require('@prisma/client');
const prisma = new PrismaClient();
module.exports = prisma;
```

Kenapa dibuat terpisah (bukan di tiap file)?

- Satu PrismaClient untuk seluruh aplikasi (singleton)
- Kalau tiap file buat new PrismaClient() → terlalu banyak koneksi DB
- Semua route tinggal require("../db") saja

Jalankan migration:

```
# Jalankan dari folder produk-api/
npx prisma migrate dev --name init
```

Yang terjadi setelah migration:

- Prisma baca schema.prisma
- Buat file SQL di prisma/migrations/
- Jalankan SQL → buat file prisma/dev.db
- Generate Prisma Client (bisa query DB dari JS)

Cek hasilnya:

```
# Buka Prisma Studio — Interface visual database
npx prisma studio --port 5555
# Buka http://localhost:5555
# Tabel "Product" harus muncul
```

index.js

```
// index.js
require('dotenv').config(); // ← HARUS PALING PERTAMA
const express = require('express');

const app = express();
const PORT = process.env.PORT || 3000;

// Middleware global — agar bisa baca req.body
app.use(express.json());

// Routes
const productsRouter = require('./routes/products');
app.use('/api/products', productsRouter);

// Global error handler — HARUS PALING BAWAH
app.use((err, req, res, next) => {
  console.error(err);
  res.status(err.status || 500)
    .json({ error: err.message || 'Server error' });
});

app.listen(PORT, () => {
  console.log(`Server jalan di port ${PORT}`);
});
```

Hal penting:

`require('dotenv').config()`

Harus baris pertama — baca .env sebelum file lain yang butuh process.env.*

`express.json()`

Middleware agar Express bisa baca body JSON dari request POST/PUT

`app.use('/api/products', ...)`

Mount router — semua endpoint di products.js automatic prefix /api/products

`Global error handler`

Parameter ke-4 = (err, req, res, next) Harus di paling bawah setelah semua app.use()

routes/products.js

```
// routes/products.js
const express = require('express');
const prisma = require('../db');
const router = express.Router();

// GET /api/products — ambil semua produk
router.get('/', async (req, res, next) => {
  try {
    const data = await prisma.product.findMany({
      orderBy: { createdAt: 'desc' }
    });
    res.json(data);
  } catch (e) { next(e); }
});

// POST /api/products — tambah produk baru
router.post('/', async (req, res, next) => {
  try {
    const { nama, harga, stok } = req.body;
    if (!nama || harga == null)
      return res.status(400).json({ error: 'nama & harga wajib' });
    const p = await prisma.product.create({
      data: { nama, harga: +harga, stok: +(stok||0) }
    });
    res.status(201).json(p);
  } catch (e) { next(e); }
});

// ... (lanjut di slide berikutnya)
module.exports = router;
```

Pola yang selalu dipakai:

- async/await — query DB bersifat asynchronous
- try/catch — capture error Prisma
- next(e) — lempar error ke global error handler
- +harga → paksa konversi ke number (dari string body)

findMany vs create:

```
prisma.product.findMany()
```

SELECT * — ambil semua baris

```
prisma.product.create()
```

INSERT — tambah baris baru

```
orderBy: { createdAt: "desc" }
```

ORDER BY createdAt DESC

```
res.status(201)
```

HTTP 201 Created — standar untuk POST

routes/products.js — lanjutan (tambahkan sebelum module.exports)

```
// GET /api/products/:id - ambil satu produk
router.get('/:id', async (req, res, next) => {
  try {
    const p = await prisma.product.findUnique({
      where: { id: +req.params.id }
    });
    if (!p) return res.status(404).json({ error: 'Tidak ada' });
    res.json(p);
  } catch (e) { next(e); }
});

// PUT /api/products/:id - edit produk
router.put('/:id', async (req, res, next) => {
  try {
    const { nama, harga, stok } = req.body;
    const p = await prisma.product.update({
      where: { id: +req.params.id },
      data: { nama, harga: +harga, stok: +stok }
    });
    res.json(p);
  } catch (e) {
    if (e.code === 'P2025')
      return res.status(404).json({ error: 'Tidak ada' });
    next(e);
  }
});

// DELETE /api/products/:id - hapus produk
router.delete('/:id', async (req, res, next) => {
  try {
    await prisma.product.delete({ where: { id: +req.params.id } });
    res.status(204).send();
  } catch (e) {
    if (e.code === 'P2025')
      return res.status(404).json({ error: 'Tidak ada' });
    next(e);
  }
});
```

Error codes Prisma:

P2025

Record tidak ditemukan saat update/delete
→ balas 404

P2002

Nilai duplicate (unique constraint)
→ balas 400 (dipakai di P2)

req.params.id:

- Dari URL /api/products/5 → req.params.id = "5" (string!)
- Pakai +req.params.id untuk konversi ke number
- findUnique → ambil tepat 1 baris berdasarkan id

Letakkan module.exports = router di PALING BAWAH setelah semua router.get/post/put/delete

Jalankan server:

```
npm run dev  
# atau: node index.js  
# Expected: "Server jalan di port 3000"
```

Checklist — test semua endpoint ini:

POST /api/products → 201 + data produk baru

GET /api/products → 200 + array produk

GET /api/products/1 → 200 + satu produk

PUT /api/products/1 → 200 + data terupdate

DELETE /api/products/1 → 204 No Content

P1 Selesai Jika :

- Server start tanpa error di terminal
- POST berhasil tambah produk → tersimpan di DB
- GET all menampilkan produk yang baru dibuat
- GET by id menampilkan satu produk saja
- PUT berhasil update data produk
- DELETE mengembalikan 204 (tidak ada body)
- Cek di Prisma Studio: data ada di tabel Product

Buka terminal baru (server tetap jalan di terminal lama), lalu jalankan urut dari atas:

1 POST — tambah produk baru

```
curl -X POST http://localhost:3000/api/products \
  -H "Content-Type: application/json" \
  -d '{"nama":"Apel","harga":5000,"stok":10}'
```

→ Expected:

```
201 + data produk:
{"id":1,"nama":"Apel","harga":5000,"stok":10,...}
```

2 GET — lihat semua produk

```
curl http://localhost:3000/api/products
```

→ Expected:

```
200 + array produk:
[{"id":1,"nama":"Apel",...}]
```

3 GET /:id — lihat satu produk

```
curl http://localhost:3000/api/products/1
```

→ Expected:

```
200 + satu produk:
{"id":1,"nama":"Apel",...}
```

4 PUT — edit produk

```
curl -X PUT http://localhost:3000/api/products/1 \
  -H "Content-Type: application/json" \
  -d '{"nama":"Apel Merah","harga":6000,"stok":8}'
```

→ Expected:

```
200 + data terupdate:
{"id":1,"nama":"Apel Merah","harga":6000,...}
```

5 DELETE — hapus produk

```
curl -X DELETE http://localhost:3000/api/products/1
```

→ Expected:

```
204 No Content
(tidak ada body di response)
```

Postman lebih nyaman untuk request dengan Body JSON. Download di postman.com/downloads

Setup di Postman:

- 1
- Buka Postman → klik tombol + untuk request baru
- 2
- Pilih METHOD (GET/POST/PUT/DELETE) di dropdown kiri
- 3
- Ketik URL: `http://localhost:3000/api/products`
- 4
- Untuk POST & PUT: klik tab Body → pilih raw → pilih JSON
- 5
- Paste JSON body (lihat contoh di kanan)
- 6
- Klik tombol Send → lihat response di bawah

Tabel semua endpoint:

METHOD	URL	Body	Expected
POST	/api/products	{ "nama": "Apel", "harga": 5000, "stok": 10 }	201
GET	/api/products	(kosong)	200
GET	/api/products/1	(kosong)	200
PUT	/api/products/1	{ "nama": "Apel Merah", "harga": 6000, "stok": 8 }	200
DELETE	/api/products/1	(kosong)	204

Yang ditambahkan di P2:

- Tabel User di database (email, password hash, role)
- Endpoint POST /api/auth/register
- Endpoint POST /api/auth/login → mengembalikan JWT token
- Password di-hash dengan bcrypt (tidak disimpan plaintext)

File yang berubah / ditambahkan:

	prisma/schema.prisma	← DIUPDATE (tambah model User)
	prisma/migrations/..._tambah_user/	← BARU (dibuat otomatis)
	routes/auth.js	← BARU (register & login)
	index.js	← DIUPDATE (tambah route /api/auth)
	.env	← DIUPDATE (tambah JWT_SECRET jika belum)

Alur Register & Login:

REGISTER

1. Terima email + password dari req.body
2. Validasi: keduanya wajib, password min 8 karakter
3. Cek apakah email sudah ada di DB
4. Hash password: `bcrypt.hash(password, 10)`
5. Simpan ke tabel User
6. Balas 201 + { id, email, role } (TANPA password)

LOGIN

1. Terima email + password dari req.body
2. Cari user berdasarkan email
3. Cek: `bcrypt.compare(password, user.password)`
4. Kalau cocok → buat JWT: `jwt.sign({userId, email, role}, secret)`
5. Balas 200 + { token, user }

prisma/schema.prisma — tambah model User + update Product

```
// Tambahkan model User (di bawah model Product yang ada)
model User {
  id          Int          @id @default(autoincrement())
  email       String       @unique
  password    String
  role        String       @default("user")
  createdAt   DateTime     @default(now())
  updatedAt   DateTime     @updatedAt
  products    Product[]
}

// Di model Product yang sudah ada,
// TAMBAHKAN dua baris ini di bagian bawah:
model Product {
  id          Int          @id @default(autoincrement())
  nama        String
  harga       Int
  stok        Int          @default(0)
  createdAt   DateTime     @default(now())
  updatedAt   DateTime     @updatedAt
  userId      Int?
  user        User? @relation(fields: [userId], references: [id])
}
```

Penjelasan relasi User ↔ Product:

- `products Product[]` → satu User bisa punya banyak Product (virtual field, tidak jadi kolom di DB)

- `userId Int?` → kolom foreign key di tabel Product (tanda ? = boleh null = boleh produk tanpa user)

- `@relation(...)` → memberitahu Prisma cara JOIN kedua tabel

- `@unique` pada email → satu email hanya bisa daftar sekali

- Setelah schema selesai → lanjut ke Step 2 (jalankan migration)

Jalankan dari folder produk-api/:

```
npx prisma migrate dev --name tambah_user
```

- Prisma otomatis buat file SQL baru di prisma/migrations/
- Tabel User dibuat di database
- Kolom userId ditambahkan ke tabel Product yang sudah ada
- Prisma Client di-regenerate agar bisa query tabel User

Verifikasi di Prisma Studio:

```
npx prisma studio --port 5555  
# Buka http://localhost:5555  
# Tabel "User" harus sudah muncul di samping "Product"
```

Kalau migration gagal → periksa kembali schema.prisma. Pastikan tidak ada typo dan relasi sudah benar.

routes/auth.js (file baru)

```
// routes/auth.js
const express = require('express');
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');
const prisma = require('../db');
const router = express.Router();

// POST /api/auth/register
router.post('/register', async (req, res, next) => {
  try {
    const { email, password } = req.body;

    // 1. Validasi input
    if (!email || !password)
      return res.status(400).json({ error: 'Email & password wajib' });
    if (password.length < 8)
      return res.status(400).json({ error: 'Password min 8 karakter' });

    // 2. Cek email sudah ada
    const sudahAda = await prisma.user.findUnique({ where: { email } });
    if (sudahAda)
      return res.status(400).json({ error: 'Email sudah terdaftar' });

    // 3. Hash password lalu simpan
    const hash = await bcrypt.hash(password, 10);
    const user = await prisma.user.create({
      data: { email, password: hash },
      select: { id: true, email: true, role: true },
    });

    res.status(201).json({ message: 'Berhasil', user });
  } catch (e) { next(e); }
});

// (login di slide berikutnya - tulis di file yang SAMA)
```

Kenapa password di-hash?

- Kalau DB bocor, password tidak bisa dibaca langsung
- "rahasia123" → "\$2a\$10\$N9qo8u..." (tidak bisa dikembalikan)
- Angka 10 = cost factor: makin tinggi makin susah di-bruteforce

select: { id, email, role }

- Pilih kolom mana yang dikembalikan dari DB
- password tidak ada di select → TIDAK dikirim ke client
- Jangan pernah kirim hash bcrypt ke response!

routes/auth.js — lanjutkan di file yang SAMA, tambah login + module.exports

```
// POST /api/auth/login
router.post('/login', async (req, res, next) => {
  try {
    const { email, password } = req.body;

    if (!email || !password)
      return res.status(400).json({ error: 'Wajib isi' });

    // 1. Cari user berdasarkan email
    const user = await prisma.user.findUnique({ where: { email } });

    // 2. Cek password — pesan error sengaja sama agar
    // penyerang tidak tahu mana yang salah
    const ok = user && await bcrypt.compare(password, user.password);
    if (!ok)
      return res.status(401).json({ error: 'Email atau password salah'
});

    // 3. Buat JWT token
    const token = jwt.sign(
      { userId: user.id, email: user.email, role: user.role },
      process.env.JWT_SECRET,
      { expiresIn: '7d' }
    );

    res.json({ token, user: { id: user.id, email: user.email, role:
user.role } });
  } catch (e) { next(e); }
});

module.exports = router; // ← WAJIB di paling bawah
```

Tentang JWT:

- `jwt.sign(payload, secret, options)`
- Payload: `userId, email, role` — data yang dimasukkan ke token
- `expiresIn "7d"` = token expired setelah 7 hari
- Token dikirim ke client → client simpan & kirim di header berikutnya

Struktur JWT (bisa dicek di jwt.io):

```
// Payload yang tersimpan di token:
{
  "userId": 1,
  "email": "budi@test.com",
  "role": "user",
  "iat": 1718900000, // issued at
  "exp": 1719504800 // expired
}
```



Cek di jwt.io: paste token → lihat payload terbaca. Payload BUKAN enkripsi, jangan taruh password di sana.

index.js — tambahkan bagian yang ditandai (sisanya tidak berubah)

```
// index.js - versi setelah P2
require('dotenv').config();
const express = require('express');

const app = express();
const PORT = process.env.PORT || 3000;

app.use(express.json());

// — TAMBAH BARIS INI (P2) —
const authRouter = require('./routes/auth');
const productsRouter = require('./routes/products');

// Auth route - public (tidak perlu token)
app.use('/api/auth', authRouter); // ← TAMBAH
// Products route - masih public di P2
app.use('/api/products', productsRouter);
// —————

app.use((err, req, res, next) => {
  console.error(err.message);
  res.status(err.status || 500)
    .json({ error: err.message || 'Server error' });
});

app.listen(PORT, () => {
  console.log(`Server jalan di port ${PORT}`);
});
```

Urutan yang benar di index.js:

- 1 `require("dotenv").config()` — PALING PERTAMA
- 2 `app.use(express.json())` — middleware global
- 3 `app.use("/api/auth", authRouter)` — public route
- 4 `app.use("/api/products", ...)` — products route
- 5 `app.use(errorHandler)` — PALING BAWAH

Test di Postman:

```
POST http://localhost:3000/api/auth/register
Body: {"email": "budi@test.com", "password": "rahasia123"}
```

Test Curl:

```
curl -X POST http://localhost:3000/api/auth/register \
-H "Content-Type: application/json" \
-d '{"email": "budi@test.com", "password": "rahasia123"}'
```

Yang ditambahkan di P3:

- Middleware authGuard — cek token JWT di setiap request
- Middleware adminOnly — cek apakah user adalah admin
- /api/products sekarang butuh token → 401 kalau tidak ada token

File yang berubah / ditambah:

 middleware/authGuard.js	← BARU (cek token JWT)
 middleware/adminOnly.js	← BARU (cek role admin)
 index.js	← DIUPDATE (pasang authGuard)

Cara pakai token di Postman:

```
Tab Headers → tambah baris baru:  
Key:   Authorization  
Value: Bearer eyJhbGciOi... (paste token dari login)
```

Buat folder middleware/ terlebih dahulu:

```
mkdir middleware
```

middleware/authGuard.js (file baru)

```
// middleware/authGuard.js
const jwt = require('jsonwebtoken');

module.exports = (req, res, next) => {
  const authHeader = req.headers.authorization;

  // Cek format: "Bearer <token>"
  if (!authHeader || !authHeader.startsWith('Bearer '))
    return res.status(401).json({
      error: 'Akses ditolak. Login terlebih dahulu.'
    });

  const token = authHeader.slice(7); // hapus "Bearer "

  try {
    // Verifikasi & decode token
    req.user = jwt.verify(token, process.env.JWT_SECRET);
    next(); // token valid → lanjut ke route handler
  } catch {
    res.status(401).json({
      error: 'Token tidak valid atau sudah expired.'
    });
  }
};
```

Cara kerja authGuard:

1

Baca header

req.headers.authorization
harus ada dan diawali "Bearer "

2

Ambil token

authHeader.slice(7) → hapus "Bearer "
ambil token-nya saja

3

Verifikasi

jwt.verify(token, secret)
kalau token dimanipulasi → lempar error

4

Simpan ke req

req.user = decoded
route handler bisa baca userId, email, role

5

Lanjut atau stop

Valid → next()
Tidak valid → 401 (route handler tidak jalan)

middleware/adminOnly.js (file baru)

```
// middleware/adminOnly.js
// Pasang SETELAH authGuard — butuh req.user yang sudah terisi

module.exports = (req, res, next) => {
  if (req.user?.role !== 'admin')
    return res.status(403).json({
      error: 'Hanya admin yang bisa akses.'
    });
  next(); // role = admin, lanjut
};
```

401 vs 403 — bedanya apa?

401 Unauthorized

Belum login / token tidak valid
→ Suruh login dulu

403 Forbidden

Sudah login, tapi role tidak cukup
→ "Kamu tidak boleh akses ini"

Cara pakai adminOnly di route spesifik:

```
// Di routes/products.js — contoh DELETE hanya untuk admin:
const adminOnly = require('../middleware/adminOnly');

router.delete('/:id', adminOnly, async (req, res, next) => {
  // authGuard sudah jalan di app.use() → adminOnly bisa langsung
  // ...
});
```

index.js — versi final (P3)

```
// index.js — versi final setelah P3
require('dotenv').config();
const express = require('express');

const app = express();
const PORT = process.env.PORT || 3000;

app.use(express.json());

// — P3: import middleware —————
const authGuard = require('./middleware/authGuard');

const authRouter = require('./routes/auth');
const productsRouter = require('./routes/products');

// /api/auth → public (tidak perlu token)
app.use('/api/auth', authRouter);

// /api/products → butuh login (authGuard)
app.use('/api/products', authGuard, productsRouter);
// —————

app.use((err, req, res, next) => {
  console.error(err.message);
  res.status(err.status || 500)
    .json({ error: err.message || 'Server error' });
});

app.listen(PORT, () =>
  console.log(`Server jalan di port ${PORT}`));
```

Test skenario (urut dari atas ke bawah):

Skenario 1 — Tanpa token

GET /api/products (tanpa header apapun)

→ 401 "Akses ditolak"

Skenario 2 — Token valid

Login dulu → copy token

GET /api/products Header: Authorization: Bearer <token>

→ 200 + array produk

Skenario 3 — Token salah

Ubah 1 karakter token lalu kirim lagi

→ 401 "Token tidak valid"

Skenario 4 — User biasa ke admin route

Login sbg user biasa → pakai token

DELETE /api/products/1 (kalau pakai adminOnly)

→ 403 "Hanya admin"

Cara buat user ADMIN:

```
# 1. Register user baru
POST /api/auth/register
Body: { "email": "admin@test.com", "password": "rahasia123" }
```

```
# 2. Buka Prisma Studio
npx prisma studio
# Buka http://localhost:5555
# Klik tabel "User"
# Klik baris admin@test.com
# Ubah kolom "role" dari "user" → "admin"
# Klik "Save 1 change"
```

```
# 3. Login ulang untuk dapat token baru (berisi role: "admin")
POST /api/auth/login
Body: { "email": "admin@test.com", "password": "rahasia123" }
# Token baru sudah berisi role: "admin"
```


Jalankanurut. Ganti <TOKEN> dengan token dari hasil login.

Skenario 1

Tanpa token → harusnya 401

```
curl http://localhost:3000/api/products
```

```
→ 401 { "error": "Akses ditolak. Login terlebih dahulu." }
```

Skenario 2

Login dulu → pakai token → harusnya 200

```
# Step 1: Login
curl -X POST http://localhost:3000/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"budi@test.com","password":"rahasia123"}'
```

```
# Step 2: Pakai token (salin dari response login)
curl http://localhost:3000/api/products \
  -H "Authorization: Bearer <TOKEN>"
```

```
curl http://localhost:3000/api/products \
  -H "Authorization: Bearer
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOiJEsImVtYWlsIjoiYn
VkaUB0ZXN0LmNvbSIsInJvbGUiOiJlc2VyIiwiaWF0IjoxNzg5ODA0MzYzLCJleHAiOi
jE3ODI0MDkxNjN9.dNfkr_OUvqJAI_DLeiW1lc06mpazI9_716m1qSD781M"
```

```
→ 200 [ {...produk...} ]
```

Skenario 3

Token sengaja dirusak → harusnya 401

```
curl http://localhost:3000/api/products \
  -H "Authorization: Bearer tokenSalahXXX123"
```

```
→ 401 { "error": "Token tidak valid atau sudah expired." }
```

Skenario 4

User biasa coba DELETE → harusnya 403

```
# Login sebagai user biasa → ambil tokennya
curl -X POST http://localhost:3000/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"budi@test.com","password":"rahasia123"}'
```

```
# Coba DELETE dengan token user biasa
curl -X DELETE http://localhost:3000/api/products/1 \
  -H "Authorization: Bearer <TOKEN_USER_BIASA>"
```

```
→ 403 { "error": "Hanya admin yang bisa akses." }
```

Ringkasan — Struktur Folder Final & Urutan Pengerjaan

Struktur folder akhir

```
produk-api/  
├── middleware/  
│   ├── authGuard.js    ← P3 (baru)  
│   └── adminOnly.js    ← P3 (baru)  
├── prisma/  
│   ├── schema.prisma   ← P1 + diupdate P2  
│   ├── dev.db          ← file database  
│   └── migrations/  
│       ├── ..._init/    ← P1  
│       └── ..._tambah_user/ ← P2  
├── routes/  
│   ├── products.js     ← P1 (buat)  
│   └── auth.js         ← P2 (baru)  
├── db.js               ← P1 (buat)  
├── index.js            ← P1 + update P2 + P3  
├── .env                ← P1  
└── package.json        ← P1
```

Urutan endpoint yang tersedia:

P1	GET	/api/products	public*	Ambil semua produk
P1	GET	/api/products/:id	public*	Ambil 1 produk
P1	POST	/api/products	public*	Tambah produk
P1	PUT	/api/products/:id	public*	Edit produk
P1	DELETE	/api/products/:id	public*	Hapus produk
P2	POST	/api/auth/register	public	Daftar user baru
P2	POST	/api/auth/login	public	Login → dapat token
P3	semua	/api/products/*	 login	Butuh Authorization header

* public di P1–P2, berubah jadi  setelah P3 dipasang